Grounded Organizational Memory via Structured Extraction and Context Graphs

Ankit Rewar
23B2176
IIT Bombay

March 8, 2026

1 Project Overview

Organizations generate large volumes of knowledge through internal communication such as emails, chat messages, and issue trackers. However, this knowledge is often scattered across systems and difficult to retrieve over time.

The goal of this project is to build a system that converts unstructured communication into a structured long-term memory graph.

The system performs the following tasks:

- Extract structured entities and claims from raw messages
- Ground every claim in verifiable evidence
- Deduplicate repeated information
- Store knowledge in a graph database
- Retrieve relevant context for user questions

2 Dataset

The dataset used in this project is the Apache Spark developer mailing list archive.

Source:

`https://mail-archives.apache.org/mod_mbox/spark-dev/`

The archive contains email discussions between developers regarding design decisions, technical implementations, and project development.

Each email includes:

- Sender
- Timestamp
- Message content
- Thread metadata

The dataset was parsed using Python's `mailbox` module.

3 System Architecture

The system pipeline consists of the following stages:

1. Dataset ingestion
2. Structured extraction
3. Deduplication and canonicalization
4. Memory graph construction
5. Retrieval and grounding
6. Visualization

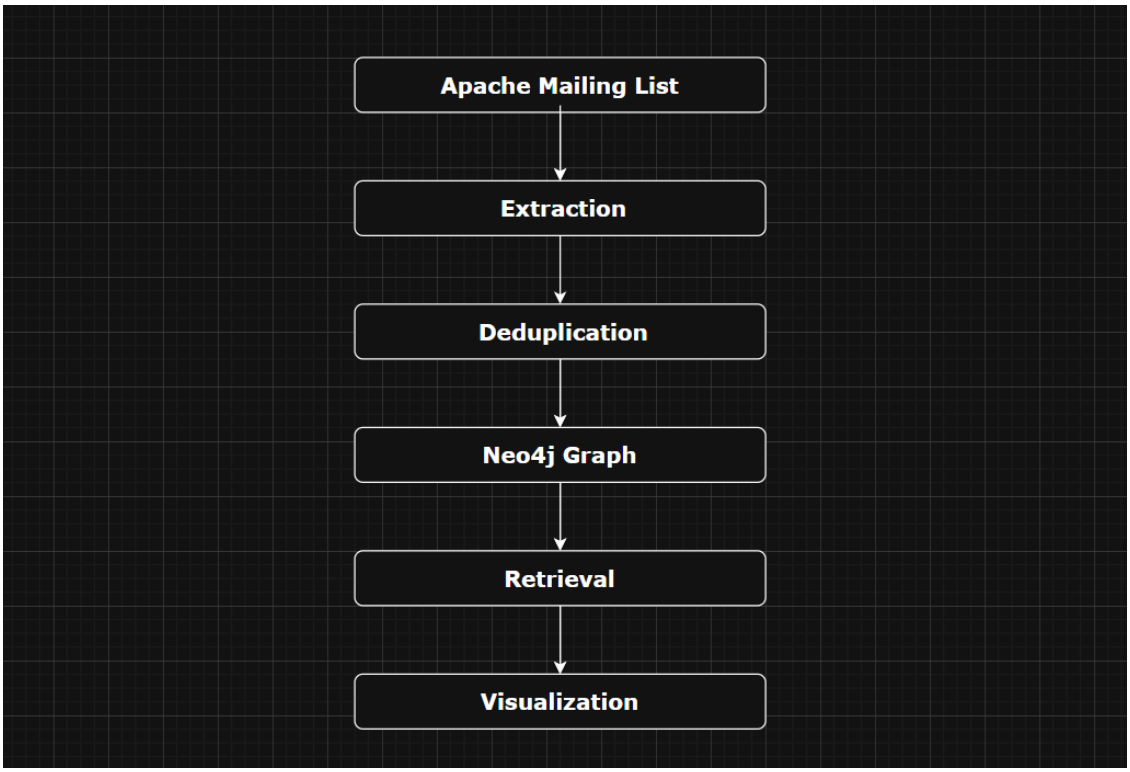

Figure 1: System Architecture

4 Ontology Design

The ontology defines entity types and relationship types used in the memory graph.

4.1 Entity Types

- Person
- Project

- Technology
- Decision
- Issue

4.2 Relationship Types

- PROPOSED
- USES
- AFFECTS
- DISCUSSES

Example claim:

$$\text{Spark} \rightarrow \text{USES} \rightarrow \text{Kafka}$$

5 Extraction Contract

The extraction system converts raw messages into structured entities and claims.

Each extracted claim must include evidence metadata.

Example extraction output:

```
{
  "subject": "Spark",
  "relation": "USES",
  "object": "Kafka",
  "evidence": "We should migrate Spark streaming to Kafka.",
  "timestamp": "2024-01-01",
  "source": "email_123"
}
```

The extraction process identifies entities, relationships, and evidence snippets.

6 Deduplication and Canonicalization

To prevent duplicated knowledge in the memory graph, deduplication occurs at multiple levels.

6.1 Artifact Deduplication

Duplicate emails are detected using message identifiers.

6.2 Entity Canonicalization

Different representations of the same entity are normalized.

Example:

- john@apache.org
- John

Both are mapped to the same entity.

6.3 Claim Deduplication

Claims are deduplicated using the tuple:

$$(\text{subject}, \text{relation}, \text{object})$$

Multiple supporting evidence sources can be attached to the same claim.

7 Memory Graph Design

The extracted knowledge is stored in a Neo4j graph database.

Graph structure:

- Nodes represent entities
- Relationships represent claims
- Relationship properties store evidence metadata

Example graph query:

```
MATCH (a)-[r]->(b)
RETURN a,r,b
```

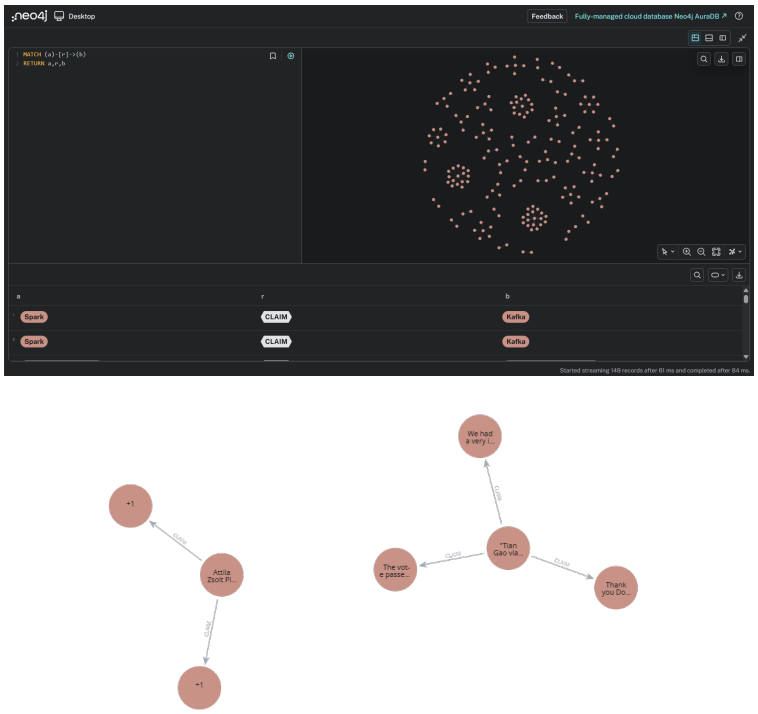

Figure 2: Example Memory Graph

8 Retrieval and Grounding

To support question answering, the system implements a semantic retrieval layer.

Steps:

1. Convert claims into embeddings using Sentence Transformers
2. Store embeddings in a FAISS vector index
3. Retrieve relevant claims for user queries

Example query:

Why did Spark adopt Kafka?

Example context pack returned:

```
Claim:
Spark USES Kafka

Evidence:
"We should migrate Spark streaming to Kafka"

Timestamp:
2024-01-01
```

9 Visualization

The system provides a visualization layer for exploring the memory graph.

Tools used:

- Neo4j Browser
- Streamlit graph viewer

Users can:

- Explore entities and relationships
- Inspect supporting evidence
- Navigate through the memory graph

10 Adapting the System to Layer10

The system can be extended to integrate organizational data sources such as:

- Slack conversations
- Jira issue trackers
- Email systems
- Internal documentation

By combining structured and unstructured data sources, the system can construct a unified long-term memory graph.

11 Reproducibility

The system can be reproduced using the following steps.

1. Download the Apache mailing list archive

   `https://mail-archives.apache.org/mod_mbox/spark-dev/`

2. Install project dependencies

   ```
   pip install -r requirements.txt
   ```

3. Configure Neo4j connection in `config.py`

4. Run the pipeline

   ```
   python main_pipeline.py
   ```

5. Open Neo4j Browser and run:

   ```
   MATCH (n)-[r]->(m) RETURN n,r,m
   ```

6. Launch visualization

   ```
   streamlit run visualization/app.py
   ```

12 Conclusion

This project demonstrates a prototype system for transforming unstructured communication into a grounded organizational memory graph.

Key contributions include:

- Structured extraction of entities and claims
- Evidence grounding
- Deduplication of repeated information
- Queryable graph storage
- Retrieval and visualization capabilities

Such systems enable organizations to preserve knowledge, understand decisions, and retrieve information efficiently.